MATH FOR MACHINE LEARNING

Complete written course • concepts, derivations, worked examples, code, quizzes and solutions

This is a detailed study document, not a one-page summary. It collects the topics and explanations available from our conversation and expands them into a step-by-step reference.

PART	CONTENTS
Represent data	Scalars, vectors, matrices, tensors, shape, rank and axes.
Linear algebra	Addition, dot products, matrix multiplication, norms and cosine similarity.
Models	Functions, features, targets, weights, bias and predictions.
Calculus	Derivatives, partial derivatives and gradients.
Loss and optimization	MSE, MAE, cross-entropy, gradient descent and learning rates.
Implementation	Linear regression from scratch with NumPy, debugging and loss curves.
Practice	Quiz review, 20 practice questions, worked answer key and formula sheet.

Study loop: read → calculate by hand → run the code → change one value → predict what will happen → explain the result aloud.

1. Mathematical objects and notation

Machine learning uses mathematics to represent data, produce predictions, measure errors and improve parameters. Learn what each symbol means in context.

Object	Meaning	Example
Scalar	One number.	3, –0.5
Vector	Ordered list of values.	[2, 5, 7]
Matrix	Rows and columns of values.	[[1,2],[3,4]]
Tensor	Array with any number of axes.	Batch of color images
Feature	Input measurement.	House size
Target	Value to predict.	House price
Parameter	Learned model value.	w or b
Prediction	Model output.	■=wx+b
Loss	Numerical error score.	MSE=2.5
Gradient	Partial derivatives for parameters.	[∂L/∂w, ∂L/∂b]

Symbols

• x or x■: input feature; y or y■: true target; ■: prediction (“y-hat”).

• w: weight; for one-feature linear regression, the slope. b: bias/intercept.

• n: number of examples. α: learning rate. L or J: loss/objective.

• ∂: partial derivative. ∇: gradient operator, collecting partial derivatives.

Prediction answers “what did the model output?” Loss answers “how wrong was it?” Gradient answers “how does loss change when parameters change?”

2. Scalars, vectors, matrices, tensors, shape and rank

A scalar is one value; a vector is an ordered list; a matrix is a two-dimensional grid; a tensor generalizes arrays to more axes. Shape gives the size along each axis. In NumPy/tensor contexts, rank means number of axes.

Object	Example	Shape	Elements
Scalar	7	()	1
Vector	[2,4,6]	(3,)	3
Matrix	2 rows, 3 cols	(2,3)	6
3-D tensor	Two 2×3 matrices	(2,2,3)	12

Number of elements = product of all shape dimensions

A grayscale image may have shape (28,28). A color image commonly uses (28,28,3) for height, width and RGB channels. A batch of 32 images may have shape (32,28,28,3); some libraries use channel-first order (32,3,28,28).

```
import numpy as np
scalar = np.array(7)
vector = np.array([2, 4, 6])
matrix = np.array([[1, 2, 3], [4, 5, 6]])
tensor = np.zeros((2, 2, 3))
print(scalar.shape)  # ()
print(vector.shape)  # (3,)
print(matrix.shape)  # (2, 3)
print(matrix.ndim)   # 2 axes
print(matrix.size)   # 6 elements
```

For tabular data with 100 examples and 4 features, X commonly has shape (100,4): rows are examples and columns are features. Targets may be (100,) or (100,1); inspect shapes because they can behave differently.

X.shape = (number of examples, number of features)

“Rank” is overloaded: tensor rank is number of axes; matrix rank in linear algebra is the dimension of its column space.

3. Vector operations and dot product

Element-wise addition

Add corresponding entries. Example: [1,2,3]+[10,20,30]=[11,22,33].

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b)  # [11 22 33]
```

Dot product

$$w \cdot x = w■x■ + w■x■ + \ldots + w_dx_d$$

For x=[2,3] and w=[4,5], multiply matching entries and add: 4×2 + 5×3 = 8+15 = 23. Two same-length vectors produce one scalar.

```
x = np.array([2.0, 3.0])
w = np.array([4.0, 5.0])
print(np.dot(w, x))  # 23.0
print(w @ x)         # 23.0
```

Why it matters for ML

$$■ = w \cdot x + b$$

Each feature contributes to the prediction according to its weight. For x=[2,3], w=[4,5], b=1, the dot product is 23 and prediction is 24.

Norm, normalization and cosine similarity

$$■x■■ = \sqrt{(x■^2 + x■^2 + \ldots + x_d^2)}$$

For x=[3,4], norm=√(9+16)=5. Divide each element by 5 to get the unit vector [0.6,0.8]. Cosine similarity compares vector directions, often for embeddings.

$$\text{cosine similarity} = (a \cdot b) / (■a■■ ■b■■)$$

```
a = np.array([3.0, 4.0])
print(np.linalg.norm(a))     # 5.0
print(a / np.linalg.norm(a)) # [0.6 0.8]
b = np.array([4.0, 3.0])
sim = np.dot(a,b)/(np.linalg.norm(a)*np.linalg.norm(b))
print(sim)                   # 0.96
```

Dot product returns a scalar; element-wise addition returns a vector. Do not confuse the operations.

4. Matrix multiplication and dataset shapes

For matrix multiplication, each output entry is a row of the first matrix dotted with a column of the second. If A is (m,n) and B is (n,p), AB is (m,p). Inner dimensions must match.

$$(m \times n) \cdot (n \times p) \rightarrow (m \times p)$$

Example A=[[1,2],[3,4]], B=[[5,6],[7,8]]. Entries: top-left 1×5+2×7=19; top-right 1×6+2×8=22; bottom-left 3×5+4×7=43; bottom-right 3×6+4×8=50.

$$AB = [[19, 22], [43, 50]]$$

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(A @ B)  # [[19 22], [43 50]]
```

Operation	NumPy	Meaning
Element-wise multiplication	A * B	Multiply corresponding entries.
Matrix product	A @ B	Row-by-column dot products.
1-D dot product	np.dot(a,b) or a @ b	Products summed into one scalar.

One earlier exercise used different matrices and produced [[31,36],[33,38]]; the requested top-left entry was 31.

$$\text{For a batch: } \blacksquare = Xw + b$$

If X has shape (100,3), w has shape (3,), and b is a scalar, Xw gives 100 predictions and broadcasting adds b to each.

5. Functions, models, weights, bias and predictions

A function maps an input to an output. A model is a function with adjustable parameters. Training changes those parameters so predictions better match target values.

f(w) = w²

At w=3, f=9; at w=–3, f=9. The minimum is at w=0, where f=0. This simple bowl-shaped function lets us study gradient descent independently of a complex model.

w	f(w)	Meaning
–3	9	Far from minimum
–1	1	Closer
0	0	Minimum
1	1	Closer
3	9	Far from minimum

Linear regression: ■ = wx + b

w controls slope and b is the intercept/vertical shift. If w=2,b=1, at x=3 prediction is 7; at x=6 prediction is 13. For multiple features, ■=w·x+b; for a batch, ■=Xw+b.

Parameters w and b are not the data. They are the values training adjusts. Inputs x are supplied to the model; the model combines x with current parameters to make ■.

6. Derivatives and the function $f(w)=w^2$

A derivative measures instantaneous rate of change; geometrically it is the slope of the tangent line. In optimization, its sign tells which direction locally increases the function and its magnitude indicates steepness.

If $f(w)=w^2$, then $f'(w)=df/dw=2w$

w	f(w)	Derivative	Interpretation
3	9	+6	Increasing w increases f locally.
–3	9	–6	Increasing w decreases f locally.
0	0	0	Flat at the minimum.
4	16	+8	Steep positive slope.

At w=4, derivative +8 means moving w positively increases f locally, so move negatively to reduce it. At w=–4, derivative –8 means moving w negatively increases f, so move positively. Subtracting the derivative gives the correct direction.

Function value and derivative are different: at w=3, f=9 is the height; f′=6 is the local slope. A zero derivative alone does not prove a global minimum for every function, but w=0 is the global minimum of w^2.

7. Partial derivatives and gradients

When loss depends on several parameters, a partial derivative measures the change with respect to one parameter while holding others fixed.

∂L/∂w = change in loss with respect to w

∂L/∂b = change in loss with respect to b

∇L = [∂L/∂θ■, ∂L/∂θ■, …, ∂L/∂θ■]■

The gradient collects one partial derivative per parameter. It points in the direction of steepest local increase; the negative gradient points toward steepest local decrease. For w and b, ∇L=[∂L/∂w,∂L/∂b].

Concept	Question answered
Prediction ■	What output does the model produce?
Loss L	How wrong is it under the chosen scoring rule?
Gradient ∇L	How would loss change if parameters moved slightly?

The gradient is calculated at current parameter values. After updating parameters, predictions, loss and gradient can all change.

8. Loss functions: MSE, MAE and binary cross-entropy

A loss converts prediction errors into a numerical score that training tries to minimize.

Mean squared error

$$MSE = (1/n)\ \Sigma_i\ (\hat{y}_i - y_i)^2$$

Subtract actual from predicted, square each error, sum, then divide by number of examples. Squaring makes errors nonnegative and penalizes large errors more strongly.

Example: y=[3,5], predictions=[2,7]. Errors=[–1,2], squared errors=[1,4], average=(1+4)/2=2.5.

```
y = np.array([3.0, 5.0])
predictions = np.array([2.0, 7.0])
mse = np.mean((predictions - y) ** 2)
print(mse)  # 2.5
```

Interpreting MSE = 305.77

It is the average squared error for the particular targets and predictions. It is not automatically good or bad by itself: its magnitude depends on target scale and units, and MSE has squared units. Compare with a baseline, target scale, and validation/test performance.

Mean absolute error

$$MAE = (1/n)\ \Sigma_i\ |\hat{y}_i - y_i|$$

MAE averages absolute error and is less sensitive to a very large error than MSE. In the example above, MAE=(1+2)/2=1.5.

Binary cross-entropy

$$BCE = -(1/n)\ \Sigma_i\ [y_i \log(p_i) + (1-y_i) \log(1-p_i)]$$

Used for binary classification when p is predicted probability of class 1 and y is 0 or 1. It penalizes confident probabilities that disagree with the true class. Implementations often clip probabilities to avoid log(0).

Low training loss does not guarantee good performance on unseen data. Validation/test data help assess generalization; overfitting occurs when a model learns patterns specific to training examples.

9. Gradient descent from first principles

Gradient descent iteratively calculates the loss gradient at current parameters, then moves in the opposite direction.

$$\theta \leftarrow \theta - \alpha \nabla L(\theta)$$

θ is one parameter or a parameter vector; α is learning rate; ∇L is the gradient at current parameters. The minus sign matters because the gradient points toward increasing loss.

One update for f(w)=w²

$$w_{new} = w - \alpha(2w) = (1 - 2\alpha)w$$

Start w=4, α=0.1. Gradient=2×4=8. New w=4–0.1×8=3.2. Old loss=4²=16; new loss=3.2²=10.24.

Stage	Calculation	Result
Current parameter	w	4
Gradient	2×4	8
Step size	0.1×8	0.8
New parameter	4–0.8	3.2
Old loss	4²	16
New loss	3.2²	10.24

```
w = 4.0
alpha = 0.1
loss_before = w ** 2
gradient = 2 * w
w = w - alpha * gradient
loss_after = w ** 2
print(loss_before)  # 16.0
print(w)            # 3.2
print(loss_after)   # 10.24
```

A loss logged before updating describes the old parameters. Recompute predictions and loss with the new parameters if you want the post-update loss.

10. Learning-rate experiments and convergence

Learning rate controls the distance moved per update. Bigger is not always better. For $f(w)=w^2$, $w_{new}=(1-2\alpha)w$.

α	Behavior	Reason
0.01	Slow convergence	Small steps.
0.1	Steady convergence	Magnitude shrinks at a useful pace.
0.5	One-step convergence	Multiplier is zero.
0.9	Oscillating convergence	Multiplier –0.8; sign flips but magnitude shrinks.
1.1	Divergence	Multiplier –1.2; sign flips and magnitude grows.

Why $\alpha=0.5$ reaches zero in one step

Substitute $\alpha=0.5$: multiplier $1-2\times0.5=0$, so $w_{new}=0\times w=0$ for any starting value. This special result depends on this exact function and update.

Derive the convergence condition

$$|1 - 2\alpha| < 1$$

This requires $-1 < 1-2\alpha < 1$. Subtract 1: $-2 < -2\alpha < 0$. Divide by –2 and reverse inequality directions: $1 > \alpha > 0$.

$$0 < \alpha < 1 \text{ (for this specific } f(w)=w^2 \text{ update)}$$

At $\alpha=1$, multiplier is –1: sign changes but magnitude stays the same, so nonzero w does not converge. At $\alpha>1$, magnitude grows and the process diverges. At $\alpha=0$, no update happens. Other loss functions can require different learning rates.

Reading a loss curve

- Steadily falling curve suggests loss is decreasing.
- Very slow fall may indicate too-small learning rate.
- Large swings or rising loss can indicate overshooting/divergence.
- For fair comparison, keep data and initial parameters fixed and change only the learning rate.

The three-question check from our conversation

1. What can happen with a large learning rate? Updates can overshoot the minimum, causing oscillation or increasing loss.

2. Start at w=4, $\alpha=0.5$. New value? Gradient=8; new $w=4-0.5\times8=0$.

3. Drawback of a very small learning rate? It may require many iterations because each step makes little progress.

You answered all three questions correctly in the earlier learning check.

11. Linear-regression gradients derived step by step

$$■■ = wx■ + b$$

$$L(w,b) = (1/n)\ Σ■\ (wx■ + b − y■)²$$

Define e■=wx■+b−y■=■■−y■. The loss is the average of e■².

Derivative with respect to w

For one example, differentiate (wx■+b−y■)² using the chain rule. The outside square contributes 2 times the inside; the inside derivative with respect to w is x■. The term is 2(wx■+b−y■)x■. Sum over examples and divide by n:

$$∂L/∂w = (2/n)\ Σ■\ x■(■■ − y■)$$

Derivative with respect to b

The inside derivative with respect to b is 1, so each term contributes 2(wx■+b−y■). Average across examples:

$$∂L/∂b = (2/n)\ Σ■\ (■■ − y■)$$

The weight gradient includes x■ because changing w changes prediction proportionally to x■. Changing b shifts each prediction by the same amount, so its gradient has no x■ factor.

Numerical check

For one example x=2,y=5,w=1,b=0: prediction=2, error=−3. Then ∂L/∂w=2×2×(−3)=−12 and ∂L/∂b=2×(−3)=−6. Subtracting negative gradients increases w and b, moving prediction upward toward 5.

12. Complete NumPy implementation

This trains one-feature linear regression from scratch; it does not use a machine-learning library to fit the model.

```
import numpy as np

# Targets follow y = 2x + 1 exactly.
X = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([3, 5, 7, 9, 11], dtype=float)

w, b = 0.0, 0.0
learning_rate = 0.01
epochs = 2000
n = len(X)
loss_history = []

for epoch in range(epochs):
    y_pred = w * X + b
    errors = y_pred - y
    loss = np.mean(errors ** 2)
    loss_history.append(loss)

    dw = (2 / n) * np.sum(X * errors)
    db = (2 / n) * np.sum(errors)

    w -= learning_rate * dw
    b -= learning_rate * db

final_predictions = w * X + b
final_loss = np.mean((final_predictions - y) ** 2)
print("Weight:", w)
print("Bias:", b)
print("Final MSE:", final_loss)
print("Prediction at x=6:", w * 6 + b)
```

Because the data follows y=2x+1, with a suitable learning rate and enough iterations w should approach 2, b should approach 1, MSE should approach zero, and prediction at x=6 should approach 13.

The code defines errors as prediction minus actual, matching the gradient formulas. If you define errors as actual minus prediction, adjust gradient and update signs consistently. Both gradients are calculated from the current parameters before both parameters are updated.

13. Loss curves and debugging

Check	Question	Why
Shapes	Do X and y align?	Mismatches may broadcast silently.
Initialization	Are values finite?	NaN/inf can break training.
Loss	Is error prediction–target?	Gradient must match the implemented loss.
Gradient	Are formulas correct?	Sign/factor errors can reverse updates.
Update	Do you subtract rate×gradient?	Gradient descent moves opposite gradient.
Scale	Are features very large?	Large values can create large gradients.
Learning rate	Exploding or slow loss?	Rate may be too large or too small.
Monitoring	Did you record loss?	Final parameters hide the training path.

```
import matplotlib.pyplot as plt
plt.plot(loss_history)
plt.xlabel("Epoch")
plt.ylabel("MSE loss")
plt.title("Training loss over time")
plt.show()
```

Try learning rates 0.001, 0.01 and 0.1 while keeping the data, initial w=0,b=0 and epochs fixed. Store each loss history and plot the curves. Feature scaling can help when feature magnitudes are large, but it does not replace checking the mathematics and learning rate.

14. Review of the earlier linear-algebra quiz

The earlier session covered vectors, matrices, tensors, dimensions, matrix multiplication, image tensor conventions and shapes. The quiz was evaluated as 4/5 essentially correct: answers 1, 2, 3 and 5 were correct. The matrix-product answer was also correct: the requested top-left entry was 31, and the full result [[31,36],[33,38]] was supplied.

• Element-wise vector addition: add matching positions.

• Dot product: multiply matching positions and sum.

• Matrix shape: (rows, columns).

• Matrix multiplication: columns of first matrix must equal rows of second; output uses outer dimensions.

• Tensor rank: number of axes in NumPy/tensor context.

• Number of elements: multiply all dimensions in the shape.

Shape	Interpretation	Elements
(5,)	Vector with 5 values	5
(2,3)	2 rows, 3 columns	6
(4,2,3)	Three axes	24
(32,28,28,3)	32 color images, 28×28, 3 channels	75,264

15. Practice problems

A. Objects and shapes

1. Is 8 a scalar, vector, matrix, or tensor?
2. What is the shape of [[1,2,3],[4,5,6]]?
3. How many elements are in shape (2,3,4)?
4. For 120 examples and 5 features, what shape is X?
5. What is the shape of A @ B if A is (4,3) and B is (3,2)?
6. Calculate [1,2,3]+[4,5,6].
7. Calculate [2,3]·[4,5].
8. For x=[3,4], calculate its norm and normalized vector.

B. Functions, derivatives and gradient descent

9. If $f(w)=w^2$, calculate $f(-3)$ and $f'(-3)$.
10. Starting at $w=4$, calculate one update with $\alpha=0.1$.
11. What does $\alpha=0.5$ do in one step for $f(w)=w^2$, and why?
12. Why can a large learning rate cause divergence?
13. What does a very small learning rate do to training time?
14. Explain the difference between loss and gradient.
15. For y=[3,5], predictions=[2,7], calculate MSE and MAE.

C. Linear regression and code

16. In ■=wx+b, what do w and b control?
17. Why does ∂L/∂w include x■ but ∂L/∂b does not?
18. What should w and b approach for data following y=2x+1 exactly?
19. Why recompute loss after updating parameters?
20. Does low training loss guarantee good unseen-data performance? Explain.

16. Worked answer key

A. Objects and shapes

• 1. Scalar: one number.

• 2. (2,3): two rows and three columns.

• 3. 2×3×4=24.

• 4. (120,5): rows are examples, columns are features.

• 5. (4,2): inner dimensions 3 match; output uses outer dimensions.

• 6. [5,7,9], element-wise addition.

• 7. 2×4+3×5=23.

• 8. Norm=$\sqrt{(3^2+4^2)}=5$; normalized vector=[0.6,0.8].

B. Functions and optimization

• 9. f(–3)=9; f′(–3)=–6 because f′(w)=2w.

• 10. Gradient=8; new w=4–0.1×8=3.2; loss goes 16→10.24.

• 11. Multiplier $1-2\alpha=0$, so w_new=0 from any start.

• 12. A step can overshoot the minimum and magnitude can grow.

• 13. It usually slows convergence because steps are small.

• 14. Loss scores current error; gradient describes how loss changes with parameters.

• 15. Errors=[–1,2]; MSE=(1+4)/2=2.5; MAE=(1+2)/2=1.5.

C. Linear regression

• 16. w controls slope/feature influence; b is the intercept/vertical shift.

• 17. Changing w changes wx■ at rate x■; changing b changes wx■+b at rate 1.

• 18. w approaches 2, b approaches 1, MSE approaches zero.

• 19. Pre-update loss describes old parameters; recompute with new parameters.

• 20. No. Overfitting is possible; validation/test data assess generalization.

17. Formula reference and complete learning loop

Topic	Formula / rule	Interpretation
Dot product	a·b=Σ■a■b■	Multiply matching entries and sum.
Matrix product shape	(m×n)(n×p)→(m×p)	Inner dimensions match.
Linear model	■=wx+b	Weight/slope plus bias.
Multiple features	■=w·x+b	Weighted sum plus bias.
MSE	(1/n)Σ■(■■–y■)²	Average squared error.
MAE	(1/n)Σ■|■■–y■|	Average absolute error.
Derivative	f(w)=w² ⇒ f′(w)=2w	Slope at current w.
Gradient descent	θ←θ–α∇L(θ)	Move opposite gradient.
Weight gradient	(2/n)Σ■x■(■■–y■)	Linear regression with MSE.
Bias gradient	(2/n)Σ■(■■–y■)	Linear regression with MSE.
Quadratic update	w_new=(1–2α)w	For f(w)=w².
Convergence	0<α<1	For this specific quadratic update.
Cosine similarity	(a·b)/(■a■■b■)	Compares vector directions.

The complete learning loop: data are represented by vectors/matrices/tensors; the model uses parameters to generate predictions; loss compares predictions with targets; derivatives/gradients describe how loss changes; gradient descent updates parameters; repetition can reduce training loss; validation and test data assess generalization.

Data → prediction → loss → gradient → parameter update → repeat → evaluate on unseen data

You understand the fundamentals when you can explain shapes, calculate a dot product, derive the simple gradients, implement the update loop without copying, interpret a loss curve and explain why learning rates behave differently.